

第四章 作业

4-5

将最优装载问题的贪心算法推广到 **2** 艘船的情形，贪心算法仍能产生最优解吗？

不能。

对于单船情形，贪心策略是把集装箱按重量从小到大排序后依次装入，这样是最优的。推广到两艘船时，一个自然的想法是先用贪心把第一艘船尽量装满，再用贪心装第二艘船。但这个策略不一定最优，下面举一个反例。

反例：设有 4 个集装箱，重量为 $w = \{2, 2, 3, 3\}$ ，两艘船载重量均为 $c_1 = c_2 = 5$

按贪心策略，排序后先装第一艘船：装入 2 和 2，总重量 4，剩余容量只有 1，装不下重量为 3 的箱子了。然后第二艘船从剩余的 $\{3, 3\}$ 中装，装入一个 3，第二个 3 装不下（ $3 + 3 = 6 > 5$ ）。所以贪心一共装了 **3** 个。

但实际上最优方案是：第一艘船装 $\{2, 3\}$ （总重 5），第二艘船也装 $\{2, 3\}$ （总重 5），这样 **4** 个全部装入。

贪心策略在第一艘船上贪心地塞了两个轻箱子，反而浪费了容量的匹配机会。本质上两艘船的装载问题包含了划分问题（NP 完全），所以简单的贪心策略无法保证最优解。

【参考代码——单船贪心装载】

```

1 // 单船最优装载：贪心选最轻的
2 int loadOneShip(vector<int>& weights, int capacity) {
3     sort(weights.begin(), weights.end());
4     int cnt = 0;
5     for (int w : weights) {
6         if (w <= capacity) {
7             capacity -= w;
8             cnt++;
9         } else break;
10    }
11    return cnt;
12 }

```

4-12

试设计一个构造图 G 生成树的算法，使得构造出的生成树的边的最大权值达到最小。

答：这个问题是最小瓶颈生成树问题，其实直接用 **Kruskal** 算法求最小生成树就行，因为最小生成树一定也是最小瓶颈生成树。

算法：就是 **Kruskal** 算法。把所有边按权值从小到大排序，依次考察每条边，如果这条边连接的两个顶点不在同一个连通分量中，就把它加入生成树，否则跳过。直到选了 $n - 1$ 条边为止。

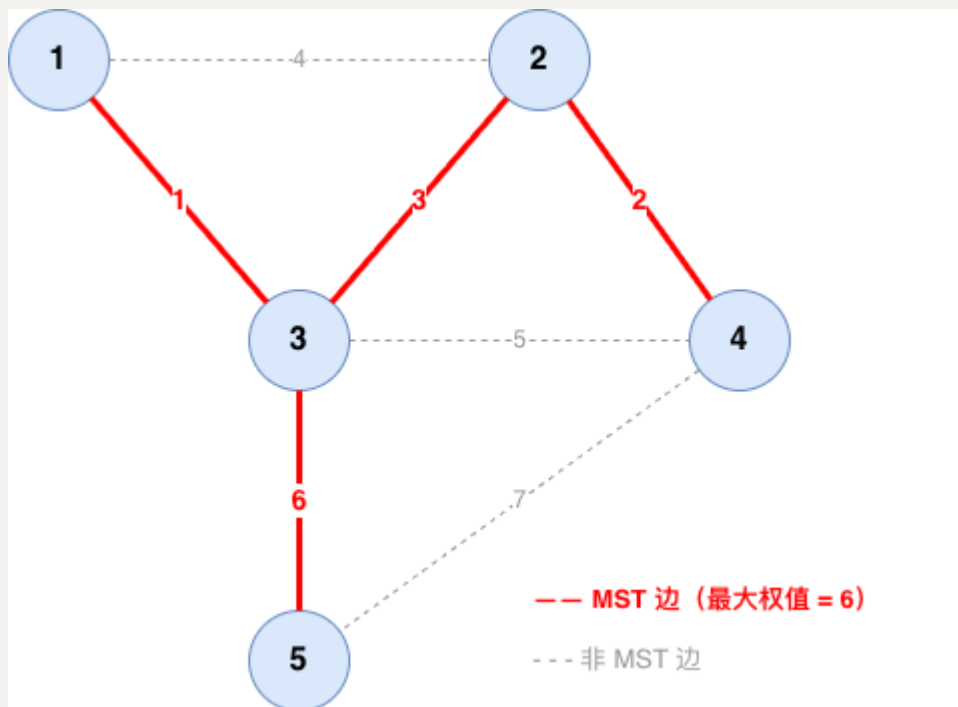
用反证法证明 MST 就是最小瓶颈生成树：设 T 是 MST，假设存在另一棵生成树 T' 的最大边权比 T 的更小。设 e^* 是 T 中最重的边，去掉 e^* 后 T 分裂成两部分 S 和 $V \setminus S$ 。 T' 中一定有一条边 e' 跨越这个分割，而且 $w(e') \leq \max(T') < w(e^*)$ 。那么把 T 中的 e^* 换成 e' ，得到的 $T - e^* + e'$ 还是生成树，但总权值更小了，这和 T 是 MST 矛盾。所以 MST 的最大边权已经是所有生成树中最小的了。

时间复杂度： $O(m \log m)$ ，主要是排序。

举个例子：下面这个图有 5 个顶点、7 条边：

Kruskal 过程：先加 $(1, 3)$ 权 1，再加 $(2, 4)$ 权 2，再加 $(2, 3)$ 权 3（此时 $\{1, 2, 3, 4\}$ 连通了）， $(1, 2)$ 权 4 跳过（已连通）， $(3, 4)$ 权 5 跳过（已连通），加 $(3, 5)$ 权 6 把顶点 5 连进来。选够 4 条边，结束。

最终生成树的最大边权是 6。而顶点 5 只有两条边（权 6 和权 7），所以任何生成树都至少包含一条权值 ≥ 6 的边，说明 6 已经是最优的了。



【参考代码——Kruskal 求最小瓶颈生成树】

```
1 struct Edge { int u, v, w; };
2
3 int find(vector<int>& p, int x) {
4     return p[x] == x ? x : p[x] = find(p, p[x]);
5 }
6
7 int minBottleneckMST(int n, vector<Edge>& edges) {
8     sort(edges.begin(), edges.end(),
9         [](Edge& a, Edge& b) { return a.w < b.w; });
10    vector<int> p(n);
11    iota(p.begin(), p.end(), 0);
12    int maxW = 0, cnt = 0;
13    for (auto& e : edges) {
14        int ru = find(p, e.u), rv = find(p, e.v);
15        if (ru != rv) {
16            p[ru] = rv;
17            maxW = max(maxW, e.w);
18            if (++cnt == n - 1) break;
19        }
20    }
```

```
21     return maxw;
22 }
```

4-14

设 G 是具有 n 个顶点和 e 条边的带权有向图，各边的权值为 $0 \sim N-1$ 之间的整数， N 为一非负整数。修改 **Dijkstra** 算法使其能在 $O(Nn + e)$ 时间内计算出从源到所有其他顶点之间的最短路径长度。

答：普通 **Dijkstra** 用二叉堆实现优先队列，复杂度是 $O((n + e) \log n)$ 。这道题边权是 $[0, N-1]$ 的整数，所以可以用桶数组来代替优先队列，这就是 **Dial** 算法的思路。

因为边权是整数且最大为 $N-1$ ，从源出发的最短路径长度最大不超过 $(n-1)(N-1) < nN$ 。所以我们开一个大小为 nN 的桶数组 $B[0..nN-1]$ ， $B[d]$ 里面放当前距离值恰好为 d 的顶点。这样取最小值只需要从当前位置往后扫描找到第一个非空桶就行，不需要堆操作。

修改后的算法：

```
1  初始化：
2      dist[s] = 0, 其余 dist[v] = ∞
3      把 s 放入 B[0]
4      ptr = 0    // 指向当前最小桶的指针
5
6  主循环：
7      重复 n 次：
8          // 找最小距离的顶点
9          while B[ptr] 为空：
10             ptr++
11             从 B[ptr] 中取出一个顶点 u
12
13             // 松弛 u 的所有出边
14             for u 的每条出边 (u, v, w):
15                 if dist[u] + w < dist[v]:
16                     如果 v 已经在某个桶里就先删掉
17                     dist[v] = dist[u] + w
18                     把 v 放入 B[dist[v]]
```

【参考代码——Dial 算法】

```
1  const int INF = 1e9;
2
3  void dial(vector<vector<pair<int,int>>>& g, int s, int n, int N)
4  {
5      vector<int> dist(n, INF);
6      dist[s] = 0;
7      int maxDist = n * N;
8      vector<list<int>> buckets(maxDist);
9      buckets[0].push_back(s);
10     int ptr = 0;
11     vector<list<int>::iterator> pos(n, buckets[0].end());
12     pos[s] = buckets[0].begin();
13
14     for (int t = 0; t < n; t++) {
15         while (ptr < maxDist && buckets[ptr].empty()) ptr++;
16         if (ptr >= maxDist) break;
17         int u = buckets[ptr].front();
18         buckets[ptr].pop_front();
19
20         for (auto& [v, w] : g[u]) {
21             if (dist[u] + w < dist[v]) {
22                 if (dist[v] != INF)
23                     buckets[dist[v]].erase(pos[v]);
24                 dist[v] = dist[u] + w;
25                 buckets[dist[v]].push_back(v);
26                 pos[v] = --buckets[dist[v]].end();
27             }
28         }
29     }
```

$O(Nn + e)$:

桶指针 `ptr` 是单调递增的，最大到 nN ，所以整个算法过程中扫描空桶的总次数不超过 nN ，这部分是 $O(Nn)$ 。每条边最多松弛一次，每次松弛是 $O(1)$ （链表的删除和插入），这部分是 $O(e)$ 。加起来就是 $O(Nn + e)$ 。

正确性和标准 **Dijkstra** 一样，因为桶数组本质上就是一个以距离为键的优先队列，`ptr` 的单调性保证了每次取出的都是当前距离最小的顶点。